第9章 一致性与复制 复习大纲

本章是分布式系统的核心内容，涉及如何通过数据复制提高系统可靠性和性能，并保证多个副本之间的一致性。内容涵盖一致性模型、复制协议、共识算法等关键技术。

一、 一致性模型（Consistency Models）

目标 ：定义数据存储系统对进程的“承诺”，规定读操作可能返回的值。

重点页码 ：文档1-P38, 文档3-P57-60

1. **以数据为中心的一致性模型（Data-Centric）**
    - **严格一致性（Strict Consistency）** ：最强模型，要求任何读操作都返回最近一次写操作的值（难以在分布式系统中实现）。
    - **顺序一致性（Sequential Consistency）** ：所有进程看到的操作顺序一致，且每个进程自身的操作顺序得到保留。**（重点）**
    - **例** ：文档3-P58图7-5，P3和P4必须看到相同的W(x)a和W(x)b的操作序列。
    - **因果一致性（Causal Consistency）** ：必须保持具有因果关系的操作顺序，并发写可以不同顺序被观察到。**（重点）**
    - **例** ：文档3-P59，若P2在读取了P1的W(x)a后执行了W(x)b，则W(x)a和W(x)b是因果关系，所有进程必须先看到a再看到b。
    - **其他** ：FIFO一致性、弱一致性等。
2. **以客户端为中心的一致性模型（Client-Centric）**
    - **单调读（Monotonic Reads）** ：一个进程对数据项X的读操作总是能看到该进程之前对X的读操作所看到的最新值或更更新的值。
    - **单调写（Monotonic Writes）** ：一个进程对数据项X的写操作必须在同一进程对该数据项的后续写操作之前完成。
    - **读己之写（Read Your Writes）** ：一个进程对数据项X的写操作完成后，该进程后续对X的读操作必须能读到这个新值或更更新的值。
    - **写后读（Writes Follow Reads）** ：一个进程对数据项X的读操作之后，同一进程对X的写操作必须在该读操作所看到的值之后生效。

二、 复制管理（Replica Management）

目标 ：通过维护数据副本提高可用性、可靠性和性能。

重点页码 ：文档3-P53-55

1. **副本类型**
    - **永久副本** ：服务器主动创建和管理。
    - **缓存副本** ：客户端主动拉取和维护。
2. **副本放置策略**
    - 考虑因素：访问模式、网络拓扑、带宽成本。
    - 常见策略：主从复制、对等复制。
3. **内容分发（Content Distribution）（重点）**

- **基于推送（Push-based）** ：服务器在数据更新时主动将更新推送给所有副本。适用于读多写少、更新不频繁的场景。
- **基于拉取（Pull-based）** ：客户端/副本在需要时主动从服务器拉取数据。适用于写多读少、更新频繁的场景。
- **对比** ：文档3-P55表格清晰展示了两种方式的差异（服务器状态、消息类型、响应时间等）。

三、 CAP理论

重点页码 ：文档3-P56

- **C（一致性）** ：所有节点在同一时间看到的数据是一致的。
- **A（可用性）** ：每个请求都能得到响应（不保证是最新数据）。
- **P（分区容错性）** ：系统在遇到网络分区时仍能继续运行。
- **定理** ：在异步网络模型中，一个分布式系统不可能同时完美满足一致性、可用性和分区容错性，只能三者取其二。
- **实践意义** ：系统设计时需要根据业务需求进行权衡。例如，CP系统（如ZooKeeper）保证一致性，AP系统（如Cassandra）保证可用性。

四、 一致性协议（Consistency Protocols）

目标 ：实现特定一致性模型的算法和机制。

重点页码 ：文档1-P36-74, 文档3-P61-66

1. **以数据为中心的一致性协议**
    - **连续一致性协议** ：通过限制数值偏差、顺序偏差和旧副本偏差来量化一致性程度。**（难点）**
    - **基于主备份的协议** ：
    - **远程写协议（Remote-Write）** ：所有写操作由主副本处理，并同步到备份副本。**（例：文档3-P61，图7-15）**
    - **本地写协议（Local-Write）** ：写操作可以转移到任何副本，该副本成为新的主副本。**（例：文档3-P62，图7-16）**
    - **复制写协议** ：
    - **主动复制（Active Replication）** ：每个副本都独立执行操作，需要全局排序器（如使用全序多播）来保证操作顺序一致。**（例：文档3-P63）**
    - **基于法定数（Quorum-based）的协议** ：**（重点）**
    - 定义读法定数（Nr）和写法定数（Nw），要求 Nr + Nw > N（总副本数），且 Nw > N/2。
    - 读操作需要从Nr个副本中读取，写操作需要更新Nw个副本。
    - **例** ：文档3-P63图7-18，RWAR（Read-One-Write-All）是Nw=N的特殊情况。
2. **以客户端为中心的一致性协议**
    - 客户端维护读集和写集，在操作前与服务器进行验证，确保单调性等约束。

五、 共识协议（Agreement Protocols）**（核心重点）**

目标 ：在存在故障的分布式系统中，使得多个节点就某个值（或状态）达成一致。

重点页码 ：文档1-全部, 文档3-P67-74

1. **Paxos（重点）**
    - **角色** ：Proposer（提议者）、Acceptor（接受者）、Learner（学习者）。
    - **阶段** ：

1. **准备阶段（Prepare）** ：Proposer获取提案权，Acceptor承诺不再接受更小编号的提案。
2. **接受阶段（Accept）** ：Proposer发送提案值，Acceptor接受。

3. **学习阶段（Learn）**：将达成共识的值传播给Learner。
   - **安全性（Safety）**：一旦值被大多数Acceptor接受（即被选定），后续任何提案都不能改变它。**（证明是关键，文档1-P16）**
   - **活性（Liveness）问题**：可能存在活锁（多个Proposer竞争）。通过选举稳定的Leader（Multi-Paxos）来避免。
   - **特点**：正确但难以理解和工程化。

2. **Raft（重点，常考）**

   - **设计目标**：易于理解，提供完整的实现基础。
   - **核心概念**：
   - **任期（Term）**：逻辑时钟，每个任期最多一个Leader。
   - **服务器状态**：Leader（领导者）、Follower（跟随者）、Candidate（候选者）。
   - **核心过程**：
1. **领导者选举（Leader Election）**：Follower超时后成为Candidate发起选举，获得多数票者成为Leader。**（文档1-P29，文档3-P71）**
2. **日志复制（Log Replication）**：Leader接收客户端请求，追加到日志，并通过AppendEntries RPC复制给Followers。**（文档1-P30-31，文档3-P71）**
   - **日志匹配属性（Log Matching Property）**：保证日志一致性。**（文档1-P32-33）**
   - **安全性**：只有日志最完整的服务器才能当选Leader。

3. **拜占庭容错协议（BFT）**

   - **问题背景**：节点可能发生任意故障（拜占庭故障），包括恶意行为。
   - **Oral Message Algorithm (OM)**：递归算法，需要 f+1 轮通信，消息复杂度指数级增长（O(n^f)）。**（文档1-P39-44，难点）**
   - **PBFT（Practical BFT）：（重点）**
   - **系统模型**：异步系统，容忍 f 个拜占庭故障，需要 n >= 3f+1 个节点。
   - **三阶段协议（正常操作）**：
1. **预准备（Pre-prepare）**：Leader分配序列号。
2. **准备（Prepare）**：节点交换消息，形成P-证书（Prepared Certificate），确保视图内顺序。
3. **提交（Commit）**：节点交换消息，形成C-证书（Committed Certificate），确保跨视图持久化。
   - **视图变更（View Change）**：处理主节点故障，保证安全性。
   - **垃圾回收**：通过检查点机制截断日志。
   - **特点**：消息复杂度为O(n^2)，真正实用化的BFT协议。

六、 重点与难点总结

- **重中之重**： **Paxos和Raft的原理、阶段、安全性保证**。务必理解Raft的选举和日志复制过程。可能出简答题或分析题。
- **核心概念**：**CAP理论**的理解和应用。可能出选择题或简答题。
- **关键区分**：**顺序一致性与因果一致性**的区别和实例判断。**基于法定数的协议**的计算（Nr, Nw）。
- **难点**：**PBFT的三阶段协议**和 **BFT Oral Message算法的递归过程**。前者要求理解消息流和证书，后者理解递归思想即可。
- **实践联系**：共识协议在ZooKeeper（Zab协议，类Paxos/Raft）、区块链（BFT类协议）等系统中的应用。

复习建议：

1. 结合文档1中的图示（如Paxos消息流、Raft状态转换、PBFT三阶段）来理解算法流程。
2. 多做文档2（样卷）中的相关题目，熟悉题型。
3. 对比学习Paxos和Raft，理解Raft如何通过引入强Leader和明确分离子问题来简化Paxos。